← 返回记忆卡 | 打印 / 另存为 PDF | ☑ 连「理解铺垫」一起印

点「打印 / 另存为 PDF」，在弹出窗口里把「目标」选为「另存为 PDF」即可保存。 每张卡的**结论**单独成块——想自测就把结论那半栏用纸盖住，看着标题先自己背一遍。 默认不印「理解铺垫」（那是理解用的长文，会让页数翻几倍）；需要就勾上面的选项。

数据结构 · 记忆卡

codebrick.tech · 用户ID 14127 · 2026-10-27 · 共 32 张

基本概念 3 张

时间空间复杂度分析 中频 硬结论数字

渐近记号含义（描述增长趋势，非具体时间）：

记号	含义
O(f(n))	渐近**上界**（最常用）： $n \geq n_0$ 时 $T(n) \leq C \cdot f(n)$
Ω(f(n))	渐近**下界**：$T(n) \geq C \cdot f(n)$
Θ(f(n))	渐近**紧界**：上界下界同阶

量级速记阶梯（从小到大背死）：

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$

- 化简两规则：加法取**较大者** O(max(f,g))；乘法**相乘** O(f·g)。只保留最高阶项、去掉系数。
- 考试默认问**最坏**情况时间复杂度（除非题目明说平均/最好）。
- **空间复杂度**度量除输入外的辅助空间；**递归算法空间 = 递归深度**（每层占常数空间时）。

时间复杂度统计的是**基本操作的执行次数 T(n)**，不是具体运行秒数——大 O 描述的是 n 足够大时的增长趋势，所以 O(n) 的算法不一定比 $O(n^2)$ 的快，只是 n 大了之后才显出差距。

怎么记不混：

- 把那条阶梯钉死，选择题里「下列复杂度排序」直接照搬。中间易漏的两档是 **O(n log n)**（夹在 n 和 n^2 之间，快排/归并/堆排在这）和 **$O(2^n)$ 在 n! 之前**。
- O/Ω/Θ 三记号：O 是「不超过」（上界），Ω 是「不低于」（下界），Θ 是「正好这个量级」（上下界同阶）。考试九成在用 O。
- 空间复杂度先想「除了输入还额外开了多少」：原地排序（冒泡/插入/选择）O(1)；归并的辅助数组 O(n)；递归看**栈深**——快排平均递归深 log n 故空间 O(log n)、最坏退化深 n 故 O(n)。

  ⚠ 给定 for 循环算出 $O(n^2)$ 还是 O(n log n)、用主定理理解 $T(n)=aT(n/b)+f(n)$——这些是**计算/推导题**（手数循环层数、套主定理），卡里不背中间步骤。卡只锚记号含义 + 量级阶梯 + 「递归空间=深度」这三条结论，演算过程交给题和可视化。

算法基本概念 冷门 定义锚点

- **算法五特性**（口诀「有确可输输」）：
  - **有穷性**——必在有限步内结束（强调「会停下来」）
  - **确定性**——每条指令含义明确、无歧义，相同输入只产生相同输出
  - **可行性**——每步都能由已实现的基本运算有限次完成
  - **输入**——有零个或多个输入
  - **输出**——至少有一个输出
- **程序 vs 算法**：算法**必须有穷**；程序**可以无穷**（如操作系统主循环）。
- **好算法四标准**：正确性、可读性、健壮性、高效率（含低存储）。

纯概念打卡点，选择题偶尔考「下列哪个不是算法特性」或「程序与算法的区别」，背下结论就拿分。

怎么记不混：

- **有穷性 vs 确定性**最易混：有穷性强调「**会停下来**」（步数有限），确定性强调「**不会有歧义**」（指令唯一确定）。一个管「终止」，一个管「明确」。
- 「**有穷性**」是算法和程序的分界线——程序可以是死循环（操作系统就靠主循环常驻），但算法定义里必须有限步终止。这一条是高频陷阱。
- 算法不等于程序：算法可以用自然语言、流程图、伪代码描述，不一定写成可运行代码。

数据结构基本概念 冷门 定义锚点

- **数据结构三要素 = 逻辑结构 + 存储结构 + 数据的运算**（缺一不可）。
- **逻辑结构**（元素间逻辑关系，与存储无关），两大类四小类：
  - 线性结构：一对一 → 线性表、栈、队列
  - 非线性结构：集合（无关系）/ 树形（一对多）/ 图状（多对多）
- **存储结构**（物理结构，逻辑结构在机内的实现）四种：
  - 顺序存储 / 链式存储 / 索引存储 / 散列存储
- **ADT（抽象数据类型）= 数据对象 + 对象上的关系 + 一组操作**；核心是封装——只管「能做什么」，不管「怎么存、怎么实现」。
- 数据层次：**数据 ⊃ 数据对象 ⊃ 数据元素 ⊃ 数据项**。

这是后面所有数据结构的总框架，本身只考概念辨析，但「三要素」和「逻辑 vs 存储」几乎年年在选择题露面。

怎么记不混：

- **逻辑结构独立于存储结构**（同一逻辑结构可顺序存也可链式存）；反过来**存储结构依赖逻辑结构**（它是逻辑结构在机内的映射）。方向别记反。
- 经典陷阱：**「有序表」属于逻辑结构**——它描述的是元素「有序」这层逻辑关系，既可顺序存也可链式存；而**「顺序表」「链表」「哈希表」是存储结构**。一字之差，考点就在这。
- 三要素里最易被忽略的是「**运算**」：二叉树和二叉排序树的逻辑结构、物理结构可以完全一样，区别只在运算定义不同——这说明运算也是不可少的要素。
- 逻辑结构四类按「关系数量」记：集合（无）→ 线性（一对一）→ 树形（一对多）→ 图状（多对多），关系越来越复杂。

线性表 2 张

顺序表 中频 对比表

顺序表 vs 链表多维对照（这门课最爱考的对比表）：

维度	顺序表（数组）	链表
随机访问（按位查找）	✓ O(1)（地址=起址+偏移）	× O(n)（从头遍历）
按值查找	O(n)	O(n)
插入/删除	O(n)，要**移动元素**（平均移 n/2）	改指针 O(1)，但**找位置仍 O(n)**
存储密度	= 1（无指针开销）	< 1（每结点多一个指针域）
空间分配	预分配，**可能溢出**/扩容需复制	动态按需申请，不溢出
适用场景	表长可估、频繁按位访问	频繁增删、表长变化大

- 一句话：顺序表赢在**随机访问 O(1)**，链表赢在**已知位置增删 O(1)**。

顺序表的本质是「逻辑相邻 = 物理相邻」，靠「起始地址 + 偏移量」一步算出任意元素地址，所以**随机访问 O(1)** 是它最大的卖点，也是链表永远做不到的。

怎么记不混：

- 链表「插删 O(1)」是有前提的——指**已知前驱位置**时只改指针 O(1)；但如果是「在第 i 位插入」，**找到第 i-1 个结点本身就要 O(n)**，整体仍是 O(n)。别被「链表插删快」误导。
- 顺序表插删「O(n)」的根源是**移动元素**：平均移 n/2 个。注意「在表尾插/删」是特例——不用移动元素，只改 length，是 O(1)。
- **存储密度**：顺序表 = 1（纯数据无额外开销）；链表 < 1（每个结点都要搭一个指针域，数据占比下降）。这是选择题常考的量化对比。
- 选结构看「查多还是增删多」：频繁按位查找选顺序表，频繁中间增删、长度不定选链表。

  ⚠️ 插入平均移动 n/2 的推导、动态扩容均摊 O(1) 的证明属于**计算/推导**，卡里只锚结论。具体移动/扩容过程看可视化或展开博客精讲。

单链表 中频 易错辨析

- **带头结点 vs 不带头结点**（408 默认带头结点）：
  - 带头结点：头结点数据域不存有效数据，**首元插删与其他位置统一处理**；空表判据 `head→next == NULL` 。
  - 不带头结点：头指针直接指首元，首元插删要特判，空表判据 `head == NULL` 。
  - 三概念别混：**头指针**（指向头结点）≠ **头结点**（不存数据的哨兵）≠ **首元结点**（第一个数据元素）。
- **建表方向**：**头插法 → 逆序**（输出与输入相反，常用于逆置）；**尾插法 → 正序**（需维护尾指针 r）。
- **找第 i 个结点 O(n)**，无随机访问，**不能用折半查找**。
- 一行带过：**双链表**=多 prior 指针、可双向遍历、插删改 4 个指针、已知结点删自身降到 O(1)；**循环链表**=尾结点指回头形成环、设**尾指针 rear** 比头指针更优。

单链表的核心是「物理位置任意、靠指针串起逻辑顺序」，所以**只能从头顺着指针走**，访问第 i 个必须遍历 i 步，这点和顺序表的随机访问是根本区别。

怎么记不混：

- **头插法逆序 / 尾插法正序**最爱考：头插法每次把新结点插到头结点之后，先来的被后来的压到后面，所以输出**逆序**（链表逆置就靠它）；尾插法每次接到表尾、维护尾指针 r，保持**正序**。
- **三个「头」别混**：头指针 → 头结点 → 首元结点。带头结点时空表是 `head→next==NULL` ；不带头结点时空表是 `head==NULL` 。选择题专在这设套。
- 带头结点的好处只有一个核心：**首元结点的插入/删除和别的位置写法一样**，不用对「第一个」单独特判，代码统一。
- 单链表**不支持随机访问**，所以折半查找用不了（折半要求顺序存储+能 O(1) 定位中点）。

  ⚠️ 插入/删除时「 `s→next=p→next` 再 `p→next=s` 」的指针顺序、逆置三指针的逐步操作属于**手练**，卡里不背步骤。只锚特性结论 + 头/尾插法效果，具体指针动作看可视化。

栈、队列与数组 4 张

栈的应用 高频 硬结论数字

① 出栈序列合法性的判据（前提：入栈序列固定为 $1, 2, \ldots, n$，入栈与出栈可交替进行）

- 栈的修改按后进先出进行，所以任何时刻**栈内元素只能按入栈的逆序弹出**。推论：若 a 比 b 先入栈却比 b 后出栈，则夹在两者之间入栈、且此刻仍压在栈中的元素，必定全部先于 a 出栈。
- **禁式判据**（与上一条等价，判合法性时更快）：合法出栈序列中不存在下标 $i < j < k$ 使 $P_j < P_k < P_i$，即不出现「大 → 小 → 中」这个三元模式；一旦找到这样的三个位置，序列必不可能由栈得到。
- 顺带钉死一条判断题结论：入栈次序确定**推不出**出栈次序确定 —— 同一入栈序列对应多个合法出栈序列。

② 出栈序列的个数

- n 个不同元素依次入栈，可能的出栈序列共 $\frac{1}{n+1}\binom{2n}{n}$ 种，即第 n 个卡特兰数；也可用递推式 $b_n = \sum_{k=0}^{n-1} b_k\, b_{n-k-1}$ 逐层推。
- 小数值当锚点（考场上直接调用，不是让你现推）：$n = 3$ 时 5 种，$n = 4$ 时 14 种。
- 同一个数还等于 n 个结点的不同二叉树棵数，两处结论可以互相印证。

③ 三类典型应用，各记一句不变量

应用	一句话不变量
括号匹配	左括号一律入栈；右括号须与栈顶左括号**同类**才弹出，否则立即判错；扫描完毕时栈空才算匹配成功
表达式求值	双栈并用 —— 运算数栈 + 运算符栈（算符优先法）；栈顶算符与当前算符比优先级，决定压栈还是取数运算，最后运算数栈顶即结果
递归 / 递归转非递归	系统设「递归工作栈」，每进一层压入一条工作记录 = 实参、局部变量、返回地址；按后调用先返回的原则弹出，栈顶恒为当前活动记录

这张卡只锚三样东西：**合法性判据、序列个数、三类应用各自的不变量**。给定一个具体序列去逐步模拟入栈出栈、中缀转后缀的逐字符转换、栈深度的逐步推演，全是纸笔手算，卡里一步都不背 —— 想练手点下方可视化逐帧走。

禁式判据为什么成立。反证即可：设出栈序列里存在 $i < j < k$ 且 $P_j < P_k < P_i$。因为入栈序列是递增的 $1, 2, \ldots, n$，P_j、P_k 都比 P_i 小，说明它们都在 P_i 之前入栈；而 P_i 却先出栈，可见 P_i 出栈的那一刻，P_j 和 P_k 都还压在栈里、位置都低于 P_i。又因为 $P_j < P_k$，P_j 入栈更早、在栈中更靠底部，于是 P_i 走后栈顶是 P_k 而不是 P_j，只能先弹 P_k 再弹 P_j —— 这与 $j < k$（P_j 排在前面）矛盾。所以「大 → 小 → 中」的三元模式一旦出现，序列必非法。判合法性时的实用价值在于：不必真的把整个过程走一遍，扫一眼有没有这种模式往往就能定性。

判据只管「1..n 顺序入栈」这一类。它成立的关键是「值小 = 入栈早」这层对应关系。若题面把入栈序列换成 a, b, c, … 之类的字母，判据同样能用，只是要把「小」理解成「入栈早」而不是字面上的小；若题面额外加了操作限制（比如不许连续三次退栈），那属于另一套条件，禁式判据管不了，得回到规则本身分析。

卡特兰数为什么和二叉树棵数是同一个数。两边的递推式长得一模一样：n 个结点的二叉树按「根 + 左子树 i 个 + 右子树 n−i−1 个」拆开累加；出栈序列按「第一个入栈的元素排在第 k+1 位出栈」拆成前 k 个和后 n−k−1 个两段独立子问题，也累加。同一个递推加同一个初值，结果自然相同。序列开头几项是 1, 1, 2, 5, 14, 42，考场上 n 较小时直接查这几项比套组合数快得多；n 较大时再用 $\frac{1}{n+1}\binom{2n}{n}$。

括号匹配的三种失败情形要分清。一是读到右括号时**栈已空**（前面的左括号都配完了，这个右括号无人认领）；二是扫描结束时**栈里还剩左括号**（有左无右）；三是右括号与**栈顶左括号类型不符**（圆方混搭）。三者都判不匹配，但触发时机不同，「最终栈空」只覆盖第二种，所以判据要三条一起看。

表达式求值别把两个栈搅在一起。操作数进运算数栈、算符进运算符栈，两者互不串门 —— 问「转换过程中运算符栈里最多同时有几个元素」时，字母一个都不算。另外「同时存在」和「累计进过栈」是两回事，栈里的元素是动态出入的，累计数总大于瞬时峰值。

递归这一格有两个常考点。其一是栈里存什么：每层的工作记录含实参、局部变量和上一层的返回地址，正因为要存这些，递归才比迭代费空间；调用嵌套时主函数在最底、最内层的调用在栈顶，出栈顺序恰好相反。其二是一条容易被记反的断言：「用非递归方式重写递归程序时必须使用栈」是**错的** —— 单向的简单递归（如求阶乘、尾递归形式）改成循环即可，用不用栈取决于递归的形态，只有存在多路分支、需要回溯保存现场时才非用栈不可。

> 出处：栈的后进先出（LIFO）定义见严蔚敏《数据结构（C 语言版）》（第 2 版）印刷 p54；括号匹配的「期待急迫程度」分析与算法（左括号入栈、右括号与栈顶同类才弹出、结束时栈空且标志为真才返回匹配成功，以及三种失败情形）见印刷 p55、p76–p77；表达式求值的算符优先法与运算数栈 OPND + 运算符栈 OPTR 双栈算法见印刷 p56、p78–p79；递归工作栈、工作记录（实参 + 局部变量 + 返回地址）、「后调用先返回」与活动记录恒在栈顶见印刷 p65–p66，章末小结重述见印刷 p83。出栈序列的禁式判据（不存在 $i < j < k$ 使 $p_j < p_k < p_i$）见殷人昆《数据结构（用面向对象方法与 C++ 语言描述）》印刷 p131 习题 3.2，2026 年真题第 42 题的标准答案亦以它作为不合法序列的等价刻画；卡特兰数公式 $b_n = \frac{1}{n+1}C_{2n}^{n}$、递推式、n=3 的 5 种出栈序列与「n 个元素进栈的出栈序列数同 n 个结点的不同二叉树棵数」见同书印刷 p209–p210（n=4 得 14 由该页递推累加 5+2+2+5 得出，2026 年真题第 42 题标答亦给出 1, 1, 2, 5, 14, 42 这一数列）。

队列 中频 对比表

栈 vs 队列（两种操作受限的线性表）：

	栈 Stack	队列 Queue
特性	**LIFO** 后进先出	**FIFO** 先进先出
操作端	同一端（栈顶）进出	队尾进、队头出
典型应用	递归（系统栈）/ 表达式求值 / 括号匹配	BFS 层序遍历 / 打印缓冲区 / 速度匹配缓冲

- 应用速记：要"撤销/回溯/嵌套"→ **栈**；要"排队/逐层/缓冲"→ **队列**。
- **双端队列（deque）**：两端都能进出，按受限方式分两类——
  - **输入受限**：只一端可入、两端都可出；
  - **输出受限**：两端都可入、只一端可出。
  - 考点 = 判断给定输出序列是否合法（识别题，不是手画指针）。

栈和队列本质都是"操作受限的线性表"——它们限制的是**逻辑上允许的操作位置**，底层既能顺序存储也能链式存储。区别只有一句：栈两端中只开一端（栈顶进出，故后进先出），队列两端各开一个（队尾进、队头出，故先进先出）。

怎么记应用不混：

- 栈对应"最近发生的先处理"——函数调用要先返回最内层（递归用系统栈）、括号匹配要先配最近的左括号、后缀表达式求值靠操作数栈，都是 LIFO 的天然场景。
- 队列对应"先来的先服务"——BFS 一层层向外扩散、打印任务按提交先后排队、主机与慢速外设之间用缓冲区抹平速度差，都是 FIFO。

双端队列的辨析是近年新增考点：普通双端队列两端全开，可退化成栈或队列；一旦给某一端加"只进"或"只出"的限制，能产生的合法输出序列就变了。考场只要求你**判断某序列能否由这种受限 deque 输出**，不需要手画入队出队的指针变化。

> 入队/出队的指针操作、出栈序列的逐步模拟、卡特兰数计数都属于手算/手练，交给题和可视化。这张卡只锚"两种结构的特性 + 应用归属 + deque 两类受限"的定性结论。

特殊矩阵压缩存储 中频 硬结论数字

① 压缩存储的定义：为多个值相同的元只分配一个存储空间，对零元不分配空间。值相同的元或零元**分布有规律**的那些矩阵才叫特殊矩阵（对称、三角、对角）；分布**没有规律**的叫稀疏矩阵。

② n 阶方阵压缩后要存多少个元素

矩阵	存储元素个数	存法
对称矩阵 $a_{ij}=a_{ji}$	$n(n+1)/2$	每一对对称元共用一格；只存上三角或下三角，**含主对角线**
三角矩阵（上 / 下）	$n(n+1)/2$ 个，再加 1 个	多出的那一格专放常数 c——三角区以外的元全等于 c（或全为零）
三对角矩阵（带状）	$3n-2$	只存主对角线及紧邻的上下两条；首、末行各 2 个，中间 $n-2$ 行各 3 个
稀疏矩阵	与非零元个数成正比	三元组表（行号, 列号, 值），存储量与 n^2 脱钩

③ 能不能随机存取，看非零元分布有没有规律

	非零元分布	存储结构	存取方式
对称 / 三角 / 三对角	有规律	一维数组，下标由公式直接算	随机存取，一步定址
稀疏矩阵	无规律	三元组表、十字链表	失去随机存取，只能顺序查

④ 两处容易漏的边角

- 三元组表除三元组之外还必须记原矩阵的行数与列数：末尾的全零行、全零列在三元组里不留任何痕迹，不记就恢复不出形状；非零元个数由表长可得。
- 邻接矩阵不是压缩存储（零元照样占位），二叉链表是二叉树的存储结构，两者都与稀疏矩阵无关。

这张卡只锚"存多少个"和"能不能随机存取"两组结论。**由 a_{ij} 求它在一维数组里的下标，是一道逐年换皮的手算题**（行优先还是列优先、上三角还是下三角、下标从 0 还是从 1，四个开关排列组合），公式记了也压不中当年的组合，卡里一步都不推——那部分交给题。

为什么能压缩，为什么只有它们能。压缩的手法只有两种：值相同的一批元素共用一格，零元干脆不给格。对称矩阵吃的是第一种，a_{ij} 与 a_{ji} 天生相等，两个元合成一格，n^2 就砍到了一半略多——之所以是"略多"而不是恰好一半，是因为主对角线上的元没有配对对象，只能自己占一格，$(n^2-n)/2+n=n(n+1)/2$ 的那个 $+n$ 就是它们。三角矩阵和三对角矩阵吃的是第二种，三角区以外、三条对角线以外的元要么是零要么是同一个常数，成片地不存。而所有这些之所以"能压"，前提是**分布有规律**：规律意味着 (i,j) 到数组下标之间存在一个可写出来的函数，取任一元素仍是一次计算加一次访存，随机存取的能力没丢。

三对角的 $3n-2$ 怎么数出来。第 1 行只有 a_{11} 和 a_{12}、第 n 行只有 $a_{n,n-1}$ 和 a_{nn}，各 2 个；中间 $n-2$ 行每行三个，$2\times 2+3(n-2)=3n-2$。首末两行少一个，是这里唯一的坑——按每行 3 个整算成 $3n$ 就多了 2 个。更一般的带宽为 b 的带状矩阵，非零元共 $(2b+1)n-b-b^2$ 个，$b=1$ 代回去正是 $3n-2$。

稀疏矩阵为什么反而"失去"了随机存取。它的零元分布没有规律，写不出下标映射函数，只能挨个把非零元连同它的行号列号一起记下来——这就是三元组表。代价是：想取 a_{ij}，没有公式可算，只能在表里顺序查有没有这一项，查不到才断定它是零。所以三元组表在**省空间**的同时**丢掉了直接存取的特性**，这一条是选择题的常客。十字链表是另一种选择，每个非零元同时挂在它所在的行链和列链上，改起来比三元组表方便，但同样不能随机定位。至于邻接矩阵，它是图的稠密存储，零元一个不落地占着空间，和"压缩"正好相反，别被"矩阵"两个字带偏。

三个特别容易搅混的边界。其一，**对称矩阵不等同于稀疏矩阵**：对称说的是取值关系，稀疏说的是零元比例，两者互不蕴含。其二，**三对角矩阵的零元比例其实和稀疏矩阵不相上下，但它不归入稀疏矩阵**——判据始终是"分布有没有规律"，不是"零多不多"。其三，稀疏与否没有精确定义，工程上常用稀疏因子 $\delta = t/(m \times n)$（t 为非零元个数）小于 0.05 作为参考线，考场上不必背这个数，只需知道它是个经验值而非定义。

关于三角矩阵那个 $+1$，两种口径都存在。主源严蔚敏的原文是"对三角矩阵进行压缩存储时，除了和对称矩阵一样，只存储其上（下）三角中的元素之外，再加一个存储常数 c 的存储空间即可"，它的下标映射式也把三角区以外的元统一指向数组末尾那一格，所以按这个口径数组长度是 $n(n+1)/2+1$，本卡采用之。也有教材只讨论对称矩阵而不单列三角矩阵，或题目直接问"三角部分有多少个元素"，那答案就是 $n(n+1)/2$，不含常量位。差别只在"常数 c 那一格算不算"，读题时看清问的是**数组要开多长 / 需要多少存储单元**（含常量位），还是**三角区里有多少个元素**（不含）。

> 出处：压缩存储的定义（"为多个值相同的元只分配一个存储空间，对零元不分配空间"）与特殊矩阵的界定见严蔚敏《数据结构（C 语言版）》（第 2 版）印刷 p100–p101；对称矩阵压缩到 $n(n+1)/2$ 个元的空间、三角矩阵"再加一个存储常数 c 的存储空间"见印刷 p101；对角矩阵与"稀疏矩阵非零元较零元少、且分布没有一定规律"见印刷 p102。对称矩阵 $n+(n-1)+\cdots+1=n(n+1)/2$ 的数法见殷人昆《数据结构（用面向对象方法与 C++ 描述）》（第 2 版）印刷 p139；三对角矩阵"总共有 $3n-2$ 个非零元素"与带状矩阵 $(2b+1)n-b-b^2$ 见印刷 p140；稀疏矩阵定义、稀疏因子 0.05、三元组 <row, column, value> 及"另外还要存储原矩阵的行数、列数和非零元素个数"见印刷 p141–p142。"三元组表存储表示下矩阵失去了随机存取的特性""三元组表结构失去了直接存取特性，只能顺序存取""三对角矩阵零元素的比例与稀疏矩阵相差不多，但不归于稀疏矩阵""对称矩阵不等同于稀疏矩阵"，以及十字链表作为稀疏矩阵的另一种存储表示，见殷人昆《数据结构习题解析》（第 2 版）印刷 p103–p104。

循环队列 冷门 硬结论数字

循环队列用取模"绕环"复用数组，难点是 `front == rear` 既可能空也可能满，必须区分。三套判空判满方案（n = MaxSize）：

方案	判空	判满	可用容量
① 牺牲一个单元（最常用）	`front == rear`	`(rear+1)%n == front`	n−1
② 设 size 计数	`size == 0`	`size == n`	n
③ 设 tag 标志	`front==rear && tag==0`	`front==rear && tag==1`	n

- **队长（牺牲单元法）= `(rear - front + n) % n`** 。
- **size 法下，队长直接 = size**（不必套上面的取模公式）。
- tag：最近一次是入队置 tag=1、出队置 tag=0；只有连续入队才会满、连续出队才会空。

循环队列把一维数组首尾"逻辑相连"，靠 `(指针+1) % n` 让 front/rear 绕环走，从而消除假溢出。它的全部考点都绕着一件事：**`front == rear` 这个等式既可能是队空也可能是队满**，三套方案就是三种区分手段。

怎么记三套方案：

- **方案①牺牲一个单元**——故意空出一格不存数据，让"满"和"空"在指针上岔开：空是 `front == rear`，满是 `rear` 再走一步就撞上 `front`，即 `(rear+1)%n == front`。代价是最多只能存 n−1 个。这是 408 默认方案。
- **方案②设 size**——直接数个数，空满一目了然，能用满全部 n 格。
- **方案③设 tag**——空满都满足 `front == rear`，靠"上一步是入还是出"打破歧义。

队长公式要分清是哪套方案：牺牲单元法（也是 size 不可用时的通用式）队长 = `(rear - front + n) % n`，加 n 再取模是为了避免 `rear < front` 时出负数；而采用 size 法时队长就是 size 本身，不需要再套取模公式（反而当队列存满时取模式会算出 0 需特判）。

> "给定 front、rear、n 绕一圈手算下标/队长"是计算题，交给题和可视化。这张卡只锚三套判空判满的公式结论 + 两种队长口径。

字符串 1 张

KMP 算法 低频 硬结论数字

KMP 模式匹配的硬结论（n=主串长、m=模式串长）：

项	结论
时间复杂度	**O(n + m)**，优于朴素 BF 的 O(n×m)
主串指针 i	**永不回溯**（失配只动模式串指针 j）
next 数组含义	`next[j]` = 模式串 T[1..j−1] 的**最长相等前后缀长度 + 1**
next/nextval	nextval 是 next 的优化：T[j]==T[next[j]] 时再跳一层，减少必失配的比较

- 失配时：`j = next[j]`，模式串右滑、主串 i 不动。
- 空间复杂度 O(m)（一个长度 m 的 next 数组）。

KMP 相对暴力匹配（BF）的全部价值，浓缩成一句：**主串指针 i 永不回溯**。BF 每次失配都把主串 i 退回起点的下一位，已比较过的字符被反复重比；KMP 用 next 数组记下"模式串自身的最长相等前后缀"，失配时只把模式串指针 j 滑到 `next[j]`，i 原地不动，于是总比较次数从 O(n×m) 降到 **O(n+m)**。

next 数组的物理含义要钉死：`next[j]` 表示当模式串第 j 个字符失配时，模式串该回退到第几个字符继续比——其值等于"第 j 个字符之前那段子串 T[1..j−1] 的最长相等前后缀长度，再加 1"。nextval 是它的优化版：若回退目标处的字符和当前字符相同，回退过去必然还是失配，于是再往前跳一层。

> ⚠ **求 next/nextval 数组的具体数值是计算题，不背**——每年必考的"手算 next 表"靠"看前缀后缀最大相等长度逐位推"现场算，背某一格的值毫无意义（且 next 有从 0 / 从 1 起两种约定，408 严蔚敏版从 1 起，须看清题目）。这张卡只锚复杂度结论、i 不回溯特性、next 的物理含义这三条定性结论；求值过程交给题和可视化。

树与二叉树 7 张

二叉树基础（性质+存储） 高频 硬结论数字

二叉树五条核心性质（死记）：

1. **$n_0 = n_2 + 1$**（叶子数 = 度2结点数 + 1，几乎每年必考）
2. 高 h 的**满二叉树**有 **$2^h - 1$** 个结点；第 i 层最多 **2^{i-1}** 个
3. n 个结点的**完全二叉树**高 **$h = \lceil \log_2(n+1) \rceil = \lfloor \log_2 n \rfloor + 1$**
4. 完全二叉树结点 i（从 1 编号）：双亲 **$\lfloor i/2 \rfloor$**、左孩子 **$2i$**、右孩子 **$2i+1$**；按 **$2i \le n$ / $2i+1 \le n$** 判有无左/右孩子
5. 满二叉树 ⊂ 完全二叉树（满二叉树是完全二叉树的特例）

完全二叉树额外结论（送分点）：

- 叶子数 = **$\lceil n/2 \rceil$**
- 度1结点 **只有 0 或 1 个**：**n 为偶数 → $n_1 = 1$，n 为奇数 → $n_1 = 0$**（且若有，只有左孩子）
- n 个结点的二叉链表有 **n + 1 个空指针域**（线索化利用的空间）

二叉树这一章命题居全章之首，性质题几乎年年出。这些公式是选择题/填空题的直接得分点，必须脱口而出——但要分清「记结论」和「手画/手算」：性质本身背，至于由这些性质算具体某棵树的结点数、画出某编号对应位置，那是计算，交给题。

几条最易踩的坑：

- **$n_0 = n_2 + 1$ 与 n_1 无关**。推导是「结点总数 $n = n_0+n_1+n_2$」和「边数 $n-1 = n_1+2n_2$」联立，n_1 被消掉了。所以无论度1结点多少个，叶子永远比度2结点多 1 个。
- **完全二叉树的度1结点只能是 0 或 1 个**。直观理解：完全二叉树按层从左到右连续排，「缺孩子」最多发生在一个结点身上——n 为偶数时恰好有一个结点只带左孩子（$n_1=1$），n 为奇数时所有非叶结点都满（$n_1=0$）。这是判断「给定结点数求叶子/分支结点数」类题的钥匙。
- **满二叉树 vs 完全二叉树别混**。满二叉树每层都填满（2^h-1 个结点）；完全二叉树只要求前 h−1 层满、最后一层从左到右连续，最后一层可以不满。满二叉树是完全二叉树的特例，反之不成立。
- **二叉树 ≠ 度为2的有序树**。二叉树严格区分左右子树、可为空；度为2的有序树至少 3 个结点、单孩子时无左右之分。

  高度从第 1 层还是第 0 层起算，不同教材有别；408 统考按「根在第 1 层」（严蔚敏约定）。具体某棵树的编号定位、由序列还原结构属计算，点「展开博客精讲」或用上方可视化手练，卡里只锚公式。

哈夫曼树与编码 高频 硬结论数字

哈夫曼树/编码硬结论：

- **WPL = Σ(叶权 × 该叶到根路径长)**；哈夫曼树是所有同权叶子的二叉树中 **WPL 最小**的。
- 等价算法：**WPL = 所有非叶结点权值之和**（合并时累加的中间和）。
- n 个叶子的哈夫曼树共 **2n − 1 个结点**（n−1 个内部结点），**无度为 1 的结点**（只有度0和度2）。
- 哈夫曼编码是**前缀编码**：任一编码都不是另一编码的前缀 → 解码无歧义；编码**不等长**（高频字符短码、低频长码）。
- **哈夫曼树形态不唯一**（左右子树可交换、等权选取有多解），但 **WPL 唯一确定**。

哈夫曼树解决的是「带权路径长度最小」问题：给一组权值（可理解为字符出现频率），构造一棵 WPL 最小的二叉树，让高频字符离根近、用短编码。这是本章命题 14 次的高频考点，常嵌在综合题里。

几条结论怎么记不混：

- **2n − 1 这个数从哪来**：构造时每次取最小两棵树合并成一棵新树（贪心），n 个叶子合并 n−1 次产生 n−1 个内部结点，总数 n + (n−1) = 2n−1。因为每个内部结点都是「两棵树合并」来的，所以**必有两个孩子，绝不存在度为1的结点**——这是判断「某棵树是不是合法哈夫曼树」的硬指标。
- **WPL = 非叶结点权值和**：每次合并产生的新结点权值 = 两子树权值和，这些中间和累加起来正好等于 WPL，比「逐叶算权×层」更快，是手算时的捷径（但具体加法是计算，不背中间值）。
- **前缀编码 = 无歧义的根**：每个字符对应一个叶结点，叶子不会是另一个叶子到根路径上的中间点，所以没有任何编码是另一个的前缀，解码时可一路唯一确定。408 常考「判断给定编码方案是否满足前缀条件」。
- **形态不唯一、WPL 唯一**：同一组权值，左右子树交换或等权时选取不同，会画出不同形态的哈夫曼树，但只要 WPL 相同都合法。选择题问「哪个不是合法哈夫曼树」，看 WPL 是否最小即可。

  手工构造合并过程、画出具体哈夫曼树、算某组权值的 WPL 数值，都是手练/计算，交给题和上方可视化。卡里只锚 WPL 定义、2n−1、无度1结点、前缀码性质这几条结论。

树与森林 中频 对比表

一般树三种存储对照：

存储方式	找双亲	找孩子	适用
双亲表示法	**O(1)**	O(n) 扫数组	查双亲为主（并查集）
孩子表示法	O(n)	**O(度)** 遍历孩子链	查孩子为主（目录展开）
孩子兄弟法	不方便	第一个 O(1)	**= 转二叉树**（左=长子、右=兄弟）

树/森林 ↔ 二叉树遍历对应（综合题直接得分点）：

树	森林	对应二叉树
先根遍历	先序遍历	**先序遍历**
后根遍历	中序遍历	**中序遍历**

- 孩子兄弟法（左孩子右兄弟）把一般树唯一映射成二叉树 → 转换天然成立。
- **树转二叉树后根无右子树**；**森林转二叉树后根有右子树**（各树根串成右链）。
- 一般树**无中根遍历**、森林**无后序遍历**。

一般树结点的度不固定（可能 1、2、几十个孩子），没法像二叉树那样用固定大小的结点存，所以要专门的存储方式。三种存储的取舍核心就一句话：**双亲表示法找双亲快、孩子表示法找孩子快、孩子兄弟法专为转二叉树**。

选存储方式的套路：408 选择题会描述一个使用场景（如「频繁查某结点的所有后代」），让你选合适的存储。先判断主操作是「找双亲」还是「找孩子」——找双亲为主选双亲表示法（并查集就用它），找孩子为主选孩子表示法，要转二叉树处理就用孩子兄弟法。

遍历对应是综合题最爱挖的坑：用「孩子兄弟法」把树转成二叉树后，原树的**先根遍历**结果 = 二叉树的**先序遍历**，原树的**后根遍历** = 二叉树的**中序遍历**。森林同理（先序↔先序、中序↔中序）。做题时可以直接对原树/森林做先根/后根遍历，不必画转换图，结果完全一致。

> 易混点：一般树没有「中根遍历」（孩子数不定，「在哪个孩子前访问根」无定义）；森林没有「后序遍历」（大纲只列先序、中序）。还要记住树转二叉树后根无右子树、森林转后根有右子树——这是判断「二叉树能还原为树还是森林」的依据。
>
> 画孩子兄弟二叉树、由遍历序列还原、数转换后结点关系都是手算，交给题和上方可视化。卡里只锚三存储取舍 + 遍历对应表。

堆 中频 硬结论数字

堆硬结论：

- 堆是满足堆序性的**完全二叉树**，用**数组顺序存储**（无需指针）。
  - **大根堆**：每个结点 ≥ 其孩子（根是最大值）
  - **小根堆**：每个结点 ≤ 其孩子（根是最小值）
- 堆**只保证双亲优于孩子**，不保证左右孩子之间、同层之间有序。
- **复杂度硬结论**：
  - **建堆 O(n)**（自底向上下沉，不是 O(n log n)！）
  - 插入 / 删除堆顶 **O(log n)**；取最值（堆顶）**O(1)**；查任意元素 **O(n)**
- **插入**：新元素放数组末尾 → **上浮**（与双亲比，大则换）。
- **删除堆顶**：取走堆顶 → 末元素补到堆顶 → **下沉**（与较大孩子换）。

堆是一种**独立的数据结构**（不只是堆排序的副产品），还用于优先队列、TopK、哈夫曼合并。它是完全二叉树，所以能用数组按层序编号存储，下标关系直接套二叉树性质（1 起：双亲 $\lfloor i/2 \rfloor$、左 2i、右 2i+1）。

最该钉死的两条结论：

- **建堆是 O(n) 不是 O(n log n)**——这是选择题/填空题最爱的陷阱。虽然单次下沉最坏 O(log n)，但叶子（约一半结点）根本不下沉、底层结点下沉路径很短，按层求和是收敛级数，总代价 O(n)。而「逐个插入上浮」式建堆才是 O(n log n)，效率不如自底向上下沉。
- **建堆 O(n) ≠ 堆排序 O(n log n)**。堆排序 = O(n) 建堆 + n 次 O(log n) 删堆顶 = O(n log n)，慢在后面 n 次删除，不在建堆。

插入 vs 删除的方向相反：插入是新元素从末尾**上浮**（往根方向，和双亲比）；删除堆顶是把末尾元素提到堆顶后**下沉**（往叶方向，和较大/较小孩子比）。为什么删除要「末尾补位再下沉」而不是「删根后合并左右子堆」——因为后者会破坏完全二叉树形态（最后一层不再连续）。

> 易混：堆 vs 二叉排序树——堆只保证双亲优于孩子，BST 保证整棵树中序有序；在堆里查任意元素是 O(n)，O(1) 只对堆顶（最值）成立。
>
> 上浮/下沉的逐步交换、建堆每一步的形态、插入删除后画出具体堆，都是手练，交给题和上方可视化。卡里只锚性质 + 复杂度结论 + 上浮/下沉方向。

由遍历序列构造二叉树 中频 易错辨析

① 四种遍历的访问次序（"先/中/后"指的是根被访问的时机）：

遍历	访问次序	定根的位置
先序 NLR	根左右	序列**首**元素 = 根
中序 LNR	左根右	根把序列劈成左、右两段
后序 LRN	左右根	序列**末**元素 = 根
层序	自上而下、同层从左到右，借队列实现	序列**首**元素 = 根

- 附带一条常用结论：**二叉排序树的中序遍历序列 = 结点值递增有序**。

② 两种遍历序列能否唯一确定二叉树（大前提：各结点的值互不相同）：

组合	唯一确定
先序 + 中序	✓
后序 + 中序	✓
层序 + 中序	✓
先序 + 后序	×
先序 + 层序	×
后序 + 层序	×

- 一句话记法：**组合里必须含中序**。先序 / 后序 / 层序都只解决"谁是根"，只有中序解决"左右怎么分"，两件事都办到才能还原。

③ × 的例外（别背成绝对结论）：先序 + 后序的歧义只来自度为 1 的结点——单孩子分不清是左是右。若树中不存在度为 1 的结点（每个非叶结点都有两个孩子，即**正则二叉树 / 严格二叉树**），先序 + 后序也能唯一确定；满二叉树只是这类树的特例，不是唯一条件。

这张卡只锚"次序"和"判据"两组结论。**由具体序列还原出树、再写出第三种遍历，是手算综合题**，属于纸笔练习，卡里一步都不背——想练手就点下方可视化逐帧走。

为什么中序是关键。任何一种遍历要还原结构，都得回答两个问题：谁是根、左右子树各包含哪些结点。先序把根放在最前、后序把根放在最末、层序把根放在最前，三者都只回答了第一个问题；只有中序把根夹在中间，根一旦确定，它左边那段必是左子树的全部结点、右边那段必是右子树的全部结点，第二个问题就被解决了。所以"能不能唯一确定"这道判断题，不必去想具体形状，只看组合里有没有中序——没有中序，左右分界永远悬着。这也解释了为什么先序 + 层序、后序 + 层序同样不行：两个都只会定根，凑一起还是定根。

中序不是万能，前提是值互不相同。有重复值时，"在中序里定位根"本身就有多个位置可选，上面所有 ✓ 都会失效。教材陈述这条结论时是带着"各结点的值均不相同"这个前提的，答题时若题面出现重复关键字，要先意识到这一层。

先序 + 后序为什么不行，以及它到底能推出什么。最小反例是先序 `AB` 、后序 `BA` ：B 挂在 A 的左边还是右边都符合，两棵不同的树。歧义的来源是"只有一个孩子的结点"——它的孩子在两条序列里的相对位置完全一样，看不出左右。反过来说，只要树里没有度为 1 的结点，歧义就消失了，这才是先序 + 后序可唯一确定的充要条件；把它记成"对满二叉树才唯一"是把条件收得太

紧了。另外，"不能唯一确定"不等于"什么都推不出"：先序去根后的首元素仍是某棵子树的根、后序去根后的末元素也是，层级关系照样能定，2012 年那道题问"根结点的孩子结点有哪些"，答案就完全确定。还有一条好用的判据：去掉根之后两条序列**完全逆序**，说明整棵树退化成一条单链（每个非叶结点只有一个孩子），2011 年那道题正是靠它把范围锁死到 8 种链形。

层序 + 中序别按位置切。先序、后序里同一棵子树的结点是**连续**的一段，可以照着中序算出的左子树长度直接切；层序不行——同一层里左右子树的结点是**交替**出现的，只能按"是否落在中序的根左侧"逐个筛。手算时更省事的说法是：**层序里最先出现的、落在当前中序区间内的那个结点，就是这棵子树的根**，不必真把子序列筛出来。2026 年那道给中序 + 层序求后序的题，用的就是这条。

别把森林的遍历名和二叉树的混起来。对应关系是两两成对的：**森林的先根（先序）遍历 ↔ 二叉树的先序**，**森林的中根（中序）遍历 ↔ 二叉树的中序**，而**树的后根遍历同样 ↔ 二叉树的中序**——注意"后根"是树一级的叫法，森林一级只有先根与中根两种，别把三个名字混成一堆。2020、2021 两道题的外壳都是森林转二叉树，遍历名一错，后面重建再准也白搭。

> 出处：三种遍历的递归定义见严蔚敏《数据结构（C 语言版）》（第 2 版）印刷 p122；层次遍历"从上到下、从左到右"借助队列、以及"由先序 + 中序或后序 + 中序均能唯一确定，由先序 + 后序不能唯一确定（附 AB / BA 反例）"见印刷 p125；二叉排序树中序得递增有序序列见印刷 p198。正则二叉树（严格二叉树）= 只含度 0 与度 2 结点的二叉树，见殷人昆《数据结构（用面向对象方法与 C++ 语言描述）》印刷 p188；③ 里"无度为 1 结点则先序 + 后序唯一"是由此推出的判据，教材未直接写成条文，考场上按"先序 + 后序一般不能唯一确定"作答，遇到题面明说"每个非叶结点都有两个孩子"时再启用例外。

线索二叉树 低频 定义锚点

标志位含义（钉死）：每个结点增设 ltag / rtag

标志	**= 0**	**= 1**
ltag	lchild 指**左孩子**	lchild 指**前驱**（线索）
rtag	rchild 指**右孩子**	rchild 指**后继**（线索）

- 一句话：**tag = 0 指孩子，tag = 1 是线索**。
- n 个结点的二叉链表有 **n + 1 个空指针域**，线索化就是把这 n+1 个空指针改成指向前驱/后继。
- **中序线索树找前驱/后继**：tag = 1 时沿线索直接取；tag = 0 时——找后继 = 右子树**最左下**结点，找前驱 = 左子树**最右下**结点。

线索二叉树是把二叉链表里「浪费掉的空指针」废物利用：n 个结点共 2n 个指针域，只有 n−1 条边占用了指针，剩下 n+1 个为空，正好拿来存遍历序列里的前驱/后继。这样遍历不再需要栈或递归。

为什么必须有 tag 标志位：指针改造后， `lchild` 既可能指左孩子、也可能指前驱，光看指针分不清。所以每个结点加 ltag/rtag 一位标志——0 表示这是真孩子、1 表示这是线索。忘记设 tag 是代码实现里最常见的错。

中序线索为什么两头都好找：中序序列里一个结点的前驱/后继，要么直接挂在线索上（tag=1），要么落在它的左/右子树的边缘。tag=0 时，后继是右子树第一个被中序访问的结点（最左下），前驱是左子树最后一个被访问的结点（最右下）。前序线索找前驱、后序线索找后继都困难（需要双亲信息，得借三叉链表），唯中序两头都 O(1)~O(h) 可达。

> 线索化的具体过程（用 pre 指针逐结点改空指针）、画出某棵树线索化后的形态，都是手练，交给题和可视化。
> 卡里只锚 tag 含义、n+1 空指针数、找前驱后继的定性规则。手练就点上方可视化。

并查集 冷门 定义锚点

并查集（不相交集合）定义锚点：

- 用**森林**表示若干**不相交集合**，每棵树是一个集合，**根是该集合的代表元**。
- **双亲表示法存储**： `parent[i]` 记元素 i 的父结点，根的 parent = −1（或指向自身）。
- **Find(x)**：沿 parent 链一路向上到根，返回根编号——两元素**根相同 ⟺ 同一集合**。
- **Union(x, y)**：分别 Find 出两根，让一个根指向另一个根，两树合并为一。
- **两大优化**：①**路径压缩**（Find 时把路径上结点直接挂到根下，树变扁）；②**按秩/按 size 合并**（矮树挂到高树下）。两者结合后单次操作**近乎常数时间**。

并查集专门解决「动态连通性」：不断地合并两个集合、查两个元素是否同属一个集合。Kruskal 算法判断加一条边会不会成环，靠的就是它。

核心就两个操作 + 两个优化：

- **Find 找根、Union 接根**。判断「x 和 y 是不是一伙的」=「Find(x) 是否等于 Find(y)」；合并就是让一个根认另一个根当爹。
- **路径压缩**：朴素 Find 最坏会让树退化成链表、每次 O(n)。压缩的做法是 Find 时顺手把路径上经过的结点全部直接挂到根下，下次再查就快。注意压缩**只改指针指向、不改集合划分**——同一集合的元素还是那些，只是树形变扁了。
- **按秩合并**：合并时让矮的树挂到高的树下面，避免树越长越高。等高时任选一个当根、秩 +1。

  ⚠ 路径压缩 + 按秩合并后，单次操作的均摊代价**近乎常数**，记到这个程度即可，不必背具体的复杂度符号（408 只要求知道「接近常数」）。

  给定操作序列手画森林状态、追踪 parent 数组变化、画压缩后的具体树形，都是手算，交给题。卡里只锚定义、两操作语义、两优化思想。

图 6 张

图基本概念 高频 硬结论数字

图的基本计数结论（握手定理 + 边数范围）：

- **握手定理**：无向图 **∑度 = 2e**（每条边贡两个度）；有向图 **∑入度 = ∑出度 = e**。
- **完全图边数**：n 顶点无向完全图 **n(n−1)/2** 条边；有向完全图 **n(n−1)** 条弧。
- **无向连通图**：至少 **n−1** 条边（=树，少于必不连通）；含回路至少 **n** 条边。
- **强连通图**（有向）：至少 **n** 条弧（构成一个环）。
- **非连通无向图**最多 **(n−1)(n−2)/2** 条边（边数超过此值必连通）。
- **生成树有 n−1 条边**；**生成森林边数 = n − 连通分量数**。
- **连通分量** = 无向图的极大连通子图；**强连通分量** = 有向图的极大强连通子图（"极大"=不能再加点/边仍保持连通）。

图的基本概念是本章命题 17 次的头号高频点，选择/填空常拿"由度求边、由边判连通"送分。把上面几组计数结论钉死，遇到题直接套，不用临场推。

怎么把这些数字记不混：

- **握手定理为什么是 2e**：无向图每画一条边，两个端点各 +1 度，所以全部度之和恰好是边数的 2 倍。有向图每条弧给弧尾 +1 出度、弧头 +1 入度，所以入度总和 = 出度总和 = 边数。这是"已知各顶点度求边数"的唯一工具。
- **完全图 n(n−1)/2 从哪来**：n 个顶点两两配对有 C(n,2)=n(n−1)/2 对，无向图每对一条边即此数；有向图每对要两条方向相反的弧，翻倍成 n(n−1)。
- **连通的边数下界 n−1**：边数最省的连通图就是树（n 个点 n−1 条边），再少一条必然断开。要"任何情况下都连通"得排除最坏布局，记审查捞回的上界 **(n−1)(n−2)/2**——把 n−1 个点连成完全图、留 1 个孤立点，是非连通的最多边数，再加一条就被迫连通。
- **生成树 vs 生成森林**：连通图的生成树固定 n−1 条边；非连通图按连通分量各生成一棵树，合起来就是生成森林，边数 = n − 连通分量数（每个分量比其顶点数少 1 条边）。
- **连通分量 vs 强连通分量**：前者针对无向图、后者针对有向图（要任意两点双向可达）；"极大"的意思是再也塞不进新顶点还保持（强）连通。

  求具体某个图的连通分量个数、画生成树、由邻接矩阵数度，都是手画/手算，交给题和上方可视化。卡里只锚握手定理、完全图边数、连通边数上下界、生成树/森林公式这几条死结论。

邻接矩阵 中频 对比表

邻接矩阵 vs 邻接表对照：

维度	邻接矩阵	邻接表
空间复杂度	**O(n²)**	**O(n+e)**
适用图	**稠密图**（边多）	**稀疏图**（边少）
判两点是否邻接	**O(1)**（直接查 A[i][j]）	**O(度)**（遍历链表）
求某点的度/出边	O(n)（扫一行/列）	出度 O(度)、入度 O(n+e)
表示唯一性	**唯一**	**不唯一**（链表结点顺序可变）

- 无向图邻接矩阵是**对称矩阵**（可只存上/下三角省一半空间）；有向图矩阵不一定对称。
- 无向图邻接表边结点数 = **2e**（每条边存两次）；有向图 = e。
- **DFS/BFS 遍历复杂度**：邻接矩阵 **O(n²)**、邻接表 **O(n+e)**。
- **DFS 用栈/递归，BFS 用队列**。

图有两种基本存储，选型取决于稀疏还是稠密。邻接矩阵用 n×n 二维数组直接映射；邻接表给每个顶点挂一条边链表。把上表当选型口诀，命题常考"稀疏图该选哪种存储""遍历复杂度是多少"。

几条对照怎么记：

- **空间为什么差这么多**：邻接矩阵不管有没有边都要占满 n×n 格子，所以恒 O(n²)；邻接表只为真实存在的边建结点，n 个顶点结点 + 边结点（无向 2e、有向 e）= O(n+e)。边稀疏（e 远小于 n²）时邻接表省得多，故**稀疏用表、稠密用矩阵**。
- **判邻接谁快**：矩阵查 A[i][j] 一步到位 O(1)；邻接表要顺着该顶点的链表找，最坏遍历整条链 O(度)。这是矩阵唯一明显占优的操作。
- **唯一性这个坑**：同一个图的邻接矩阵是确定的（行列固定），但邻接表里同一顶点的边链表可以任意排序，所以**表不唯一**——这直接导致 DFS/BFS 的访问序列也不唯一。选择题常给一个序列问"能否由某种邻接表产生"。
- **遍历复杂度配对存储记**：用矩阵存就 O(n²)（每个顶点都要扫一整行找邻接点），用表存就 O(n+e)（每个顶点和每条边各访问一次）。DFS 借栈/递归往深走，BFS 借队列逐层扩，这点别和具体遍历序列混。

  由邻接矩阵手画邻接表、跑一遍 DFS/BFS 写出访问序列、数某顶点的度，都是手练，交给题和上方可视化。卡里只锚空间/适用/判邻接/唯一性这张对照表 + 遍历复杂度结论。

关键路径 中频 硬结论数字

关键路径硬结论（AOE 网）：

- **AOE 网**（Activity On Edge）：**边表示活动**（权=持续时间）、顶点表示事件的有向无环图。
- **关键活动判据：e(i) = l(i)**（活动最早开始时间 = 最晚开始时间，即**时间余量 d = l − e = 0**）。
- **关键路径 = 从源点到汇点的最长路径**，由全部关键活动构成；其长度 = 工程最短工期 = ve[汇点]。
- **关键路径可能不唯一**（多条等长最长路径）。
- **只有缩短关键活动才可能缩短工期**；缩过头关键路径会变（原非关键路径升为新瓶颈），需重新计算。
- 关键路径不唯一时，要缩工期须缩**所有关键路径公共**的关键活动。

关键路径回答"这个并行 + 串行的工程最快多久干完、卡在哪些活动上"。在 AOE 网（边是活动、权是耗时）里，从开工到完工的最长那条路就是关键路径，路上的活动一拖、全工程就拖。这是连续命题、近年常和拓扑拼大题（2025-42）的高频点，背死下面几条定性结论即可。

几条结论怎么记不混：

- **AOE 网的"E"是边**：A**O**E = Activity **O**n **E**dge，**边 = 活动**、顶点 = 事件（"某些活动完成、另一些可开始"的状态）。和 AOV 网（顶点是活动）正好相反，这是最爱考的辨析。
- **关键活动判据 e=l**：每个活动有最早开始 e 和最晚开始 l 两个时刻，二者之差 d = l − e 是"不影响工期能拖多久"的余量；**余量为 0（e=l）的活动就是关键活动**，一点都拖不得。
- **为什么是最长路径**：工程必须等所有并行分支都干完才算结束，所以总工期由最慢的那条路（最长路径）决定——这和最短路径问题的思路恰好相反。关键路径长度 = 汇点最早发生时间 ve[汇点] = 最短工期。
- **缩工期的陷阱（高频）**：想让工程提前，**只能缩关键活动**（缩非关键活动只是吃掉它的余量，工期不变）；但关键活动缩过头后，原来的关键路径可能不再最长，**别的路径升级成新瓶颈**，所以要重新算。若有多条关键路径，必须缩它们**公共**的关键活动才保证整体提前。

  ve/vl/e/l 四个量的具体数值、判断哪条是关键路径、算时间余量，都是手算（计算题），交给题和上方可视化。卡里只锚 AOE 定义、e=l 判据、最长路径、可能不唯一、缩关键活动才缩工期这几条定性结论。

Dijkstra 最短路径 中频 对比表

最短路径三算法对照：

维度	Dijkstra	Floyd	BFS
问题	**单源**最短路	**多源**（任意两点）	无权图最短路
时间复杂度	**$O(n^2)$**	**$O(n^3)$**	O(n+e)
思想	贪心	动态规划	逐层扩展
负权边	**不能有**	**可有**（但不能有负权回路）	不涉及（无权）

- **Dijkstra 不能处理负权边**：一旦顶点被确定，其 dist 不再改变，负权边会破坏这个前提。
- **Floyd 可处理负权边，但不能有负权回路**（否则最短路可无限减小、无意义）。
- **BFS 求无权图（边权全 1）最短路**，即 Dijkstra 在无权图上的退化。
- 对所有顶点各跑一次 Dijkstra 也是 $O(n^3)$，与 Floyd 同阶，但 Floyd 代码更简洁且支持负权。

最短路径分单源和多源：Dijkstra 求"某一点到其余各点"，Floyd 求"任意两点之间"，无权图直接用 BFS。把上表的复杂度和负权限制钉死，这是选择题的高频区。

几条结论怎么记：

- **单源 vs 多源**：Dijkstra 从一个源点出发，每轮贪心选当前 dist 最小的未确定顶点、确定它、再用它松弛邻居，$O(n^2)$；Floyd 三重循环逐个引入中转顶点，一次性把所有顶点对的最短距离都算出来，$O(n^3)$。
- **Dijkstra 为什么禁负权边**：它的正确性建立在"被确定的顶点 dist 不会再变小"上（贪心前提）；若存在负权边，后续可能绕负边找到更短路，已确定的值就错了。注意**负权 ≠ 非正权**——Dijkstra 能处理 0 权边。
- **Floyd 的负权底线**：允许负权边，但**不能有负权回路**，否则绕圈一次比一次短、最短路无意义（判据：算法结束后若 dist[i][i]<0 即存在负权回路）。
- **BFS 是无权特例**：边权全为 1 时，按层扩展第一次到达某点就是最短，等于 Dijkstra 退化版。
- **n 次 Dijkstra vs Floyd**：要求所有点对最短路，对每个源各跑一次 Dijkstra 也是 $O(n^3)$，与 Floyd 同阶；Floyd 胜在代码更短、且能吃负权边。

  手算松弛、逐轮填 dist[] 表、回溯具体最短路径，都是计算/手练，交给题和上方可视化。卡里只锚单源/多源、复杂度、负权限制这几条对照结论。

Prim 最小生成树 中频 对比表

Prim vs Kruskal（两种最小生成树 MST 算法）：

维度	Prim	Kruskal
策略	**从点扩展**（逐顶点生长）	**从边扩展**（逐边选取）
每步动作	选连接树与树外的**最小权边**	选全局**最小权边**，不成环就要
判环	靠"是否已在树内"	**并查集**（Find/Union）
时间复杂度	**$O(n^2)$**	**O(e log e)**
适用图	**稠密图**（边多）	**稀疏图**（边少）

- 两者**都求 MST**，结果都是 **n−1 条边**、总权值最小。
- **MST 树形可能不唯一**（有等权边时），但**总权值唯一确定**。
- Prim 复杂度 $O(n^2)$ **只与顶点数有关、与边数无关**。

最小生成树有两条经典路线：Prim 从一个点出发把树"长大"，Kruskal 把所有边排好序从小往大挑。选型口诀就一句——**稠密用 Prim、稀疏用 Kruskal**，背后是两者复杂度对边数的依赖不同。

怎么记不混：

- **从点 vs 从边**：Prim 维护一个已选顶点集合，每轮挑一条"连接集合内与集合外的最小边"把新顶点拉进来，共 n−1 轮；Kruskal 不管连通性，先把所有边按权排序，从最小的开始挑，只要加进去不成环就保留，挑够 n−1 条就停。
- **复杂度对边数的依赖决定选型**：Prim 朴素实现是两层 O(n) 循环 = **$O(n^2)$，只看顶点数**，边再多也不变，所以**稠密图**它不吃亏；Kruskal 瓶颈在排序 **O(e log e)，盯着边数**，边少时排序快，所以**稀疏图**它划算。
- **Kruskal 为什么要并查集**：判断"加这条边会不会成环"等价于"两个端点是否已在同一连通块"——并查集的 Find 查根、Union 合并正好干这事，两端点同根就跳过。
- **唯一性这个高频坑**：当图里有权值相同的边时，MST 的**树形可以不唯一**（2018-42 就是两种方案），但无论哪棵树，**总权值一定相同**。选择题问"MST 是否唯一"，看有没有等权边参与竞争即可。

  Prim/Kruskal 具体跑一遍、写出选边顺序、画出 MST，都是手练，交给题和上方可视化。卡里只锚从点/从边、复杂度、稠密/稀疏选型、唯一性这几条结论。

拓扑排序 中频 定义锚点

拓扑排序核心定义：

- **AOV 网**（Activity On Vertex）：**顶点表示活动**、有向边表示活动间先后（先修）关系的有向图。
- **拓扑序列** = 每次取一个**入度为 0** 的顶点输出、删去它及其出边，重复至空。
- **前提：必须是 DAG（有向无环图）才有拓扑序列**；有环则排不出。
- **能拓扑排序 ⟺ 图无环**：故拓扑排序可用来**判断有向图是否有环**（输出顶点数 < n 即有环）。
- **拓扑序列可能不唯一**：某时刻有多个入度为 0 的顶点时，选谁先输出都合法。

拓扑排序解决"带先后约束的活动该按什么顺序做"——先修课、工序依赖都是这类问题。把先后关系建成 AOV 网（顶点是活动），找一个不违反任何"先 A 后 B"约束的线性顺序。核心就背 AOV 定义 + 入度为 0 法 + "能拓扑 ⟺ 无环"。

几条结论怎么记：

- **AOV 网的"V"是顶点**：A**O**V = Activity **O**n **V**ertex，顶点 = 活动、边 = 依赖。别和 AOE 网（边表示活动、用于关键路径）搞混，这是高频辨析点。
- **入度为 0 法的直觉**：入度为 0 = 没有任何前置活动没做完，所以可以马上做（输出）；做完它就删掉它发出的边，使后继的入度减 1，让新的"无前置"活动浮现。重复这个过程就排出了序列。
- **为什么必须 DAG**：若图里有环，环上每个顶点都互相依赖、入度永远减不到 0，谁都排不出来——这正好反过来给了**判环**的办法：跑一遍拓扑排序，最终输出顶点数小于 n 就说明有环。
- **不唯一的条件**：当某一步同时有多个入度为 0 的顶点时，先输出哪个都合法，于是产生多条拓扑序列。选择题常给一个 DAG 和几个序列，问"哪个不是合法拓扑序列"——只需检查是否违反了某条有向边的先后约束。

  给定 DAG 手工跑出某条具体拓扑序列、数共有几种合法序列，都是手练/计算，交给题和上方可视化。卡里只锚 AOV 定义、入度为 0 法、DAG 前提、"能拓扑 ⟺ 无环"、不唯一这几条结论。

查找 7 张

B 树 高频 硬结论数字

m 阶 B 树定义结论（背阶的约束数字）：

维度	根结点（非叶时）	非根结点
孩子（子树）数	2 ~ m	**⌈m/2⌉ ~ m**
关键字数	1 ~ m−1	**⌈m/2⌉−1 ~ m−1**

- **关键字数 = 孩子数 − 1**（每个结点）。
- **所有叶结点都在同一层**（绝对平衡）；这里"叶结点"指查找失败到达的**外部空结点（NULL）**，本身不含信息；有关键字的最底层结点叫"终端结点"。
- 结点内关键字**有序**（$K_1<K_2<\ldots$），结点内可顺序/折半查找。
- **为降低磁盘 I/O 而生**（多路平衡）：一次 I/O 读一个结点，查找的 **I/O 次数 = 树高 h**。
- B 树长高的**唯一方式 = 根结点分裂**。

B 树是为**磁盘访问**设计的多路平衡查找树：数据库索引动辄百万条，不可能全放内存，于是让每个结点存多个关键字、一次 I/O 读一整个结点，把树压得很矮（1001 阶存 10 亿关键字只要 3 层）。这张卡只背「阶 m 的约束数字」，插入分裂、删除合并的过程交给题。

最该钉死的三组数字/辨析：

- **非根结点的下界 ⌈m/2⌉**：孩子数至少 ⌈m/2⌉、关键字至少 ⌈m/2⌉−1。这是"半满"约束，保证空间利用率。**根结点是例外**：非叶时至少 2 棵子树、至少 1 个关键字。5 阶为例：根至少 2 子树、非根至少 3 子树——这俩最容易记混。
- **"叶结点"是外部空结点**：B 树说"所有叶结点在同一层"指的是那些查找失败到达的 NULL，不是有关键字的最底层结点（那叫终端结点）。算高度时注意题目是否把失败层计入。
- **408 用中国教材的"阶"定义**：m 阶 = 最多 m 棵子树。别被英文资料（用最小度数 t 定义）带偏。

  插入溢出后的分裂、删除下溢后的借/合并、给定序列建 B 树的具体形态，全是手练，交给题和上方可视化。卡里只锚阶 m 的孩子数/关键字数约束 + 「同层 + 为 I/O 而生」两条性质。

平衡二叉树（AVL） 中频 硬结论数字

平衡二叉树（AVL）硬结论：

- **平衡因子 BF = 左子树高 − 右子树高**，AVL 要求每个结点 **BF ∈ {−1, 0, 1}**（即 |BF| ≤ 1）。
- **四种旋转（背触发条件，不手画）**：
  - **LL**（右单旋）：失衡结点的**左孩子的左子树**插入致失衡。
  - **RR**（左单旋）：失衡结点的**右孩子的右子树**插入致失衡。
  - **LR**（先左旋后右旋）：失衡结点的**左孩子的右子树**插入致失衡。
  - **RL**（先右旋后左旋）：失衡结点的**右孩子的左子树**插入致失衡。
  - 口诀：**两同名一单旋（LL/RR），一内一外双旋（LR/RL）**。
- **高度 O(log n)**：含 n 结点的 AVL 树高恒为 O(log n)，最坏也稳定。
- **高度为 h 的 AVL 最少结点数递推**：**N(h) = N(h−1) + N(h−2) + 1**，其中 N(0)=0、N(1)=1（类斐波那契）。

BST 会退化成链，AVL 的解法很暴力：**每次插入/删除后沿路径回溯检查平衡因子，一旦 |BF| > 1 就旋转掰直**。代价是每结点多存一个高度，换来稳定的 O(log n)。这张卡只背平衡因子定义、四种旋转的触发条件、两条数字结论，旋转的具体构造过程交给题。

最该钉死的三组结论：

- **平衡因子定义方向**：BF = 左高 − 右高（不是右减左），取值只能是 −1/0/1，超出就失衡要调整。
- **四种旋转认门**：先看失衡结点哪边沉（左沉还是右沉），再看是「同方向延伸（LL/RR，单旋一次）」还是「拐了个弯（LR/RL，双旋两次）」。记触发条件即可，旋转怎么转交给手练。
- **最少结点数递推 N(h)=N(h−1)+N(h−2)+1**：这是「给定高度 h 问最少多少结点」或反过来「n 个结点 AVL 最大高度」这类题的钥匙，套递推即得。

  构造 AVL（逐个插入 + 旋转复衡）、画出旋转后的具体形态，都是手练，交给题和上方可视化。卡里只锚平衡因子定义 + 四旋转触发条件 + 高度/最少结点数两条数字结论。

二叉搜索树 中频 易错辨析

二叉排序树（BST）性质 + 易错辨析：

- **结构性质（递归定义）**：左子树所有结点 **< 根 <** 右子树所有结点；左右子树本身也是 BST。
- ⭐ **中序遍历得递增有序序列**（判断题最爱考）。
- **复杂度**：查找/插入/删除 **平均 O(log n)**，**最坏退化为单支链表 O(n)**（输入序列本身有序时退化）。
- **插入**：新结点一定挂成**叶子**；不允许重复关键字。
- **删除分三情况**（设待删结点 z）：
  - ① z 是叶子 → 直接删。
  - ② z 只有一棵子树 → 用该子树顶替 z。
  - ③ z 有两棵子树 → 用**中序前驱（左子树最大）或中序后继（右子树最小）**的值替换 z，再转而删那个前驱/后继（它必转化为情况①或②）。
- 同一组关键字、不同插入顺序 → 不同树形，但**中序遍历结果相同**。

BST 想兼得「数组查得快、链表插得快」：平均情况查找和插入都是 O(log n)——**前提是树别退化成链**。这张卡只背性质和退化结论，插入删除的具体画图交给题。

最该钉死的两条辨析：

- **中序遍历 = 升序**：这是 BST 的定义直接推论，也是判断题高频点。反过来，「给一棵 BST 做中序遍历就得到排序结果」也成立。
- **最坏退化 O(n)**：按 1,2,3,4,5 这种有序序列插入，每个新结点都挂到最右下，整棵树歪成一条右斜链，查找退化到和顺序查找一样 (n+1)/2 = O(n)。这正是后面引入 AVL / 红黑树自平衡的动机。

删除度为 2 的结点是大题高频子任务：核心是「找中序前驱或后继来顶替」。两种选法都对，但得到的树不同——题目指定哪种就用哪种，没指定一般默认中序前驱。

> 根据插入序列画 BST、删除后的具体形态、求某棵 BST 的 ASL，都是手算，交给题和上方可视化。卡里只锚性质 + 退化结论 + 删除三情况的「替身规则」。

各查找算法比较 中频 硬结论数字

各查找算法对照（等概率，n 个元素）：

方法	成功 ASL	失败 ASL	时间复杂度	适用结构
顺序查找	**(n+1)/2**	**n+1**（有序表可提前终止）	O(n)	顺序/链式、有序或无序
折半查找	**判定树上各成功结点层数的平均**（须画判定树算，不背公式）	判定树外部（失败）结点层数的平均	$O(\log_2 n)$	**有序 + 顺序存储**（链表不可折半）
分块查找	介于顺序与折半之间	—	$O(\sqrt{n})$（最优块长 $\sqrt{n}$）	分块有序（块间有序、块内无序）
散列查找	取决于装填因子 α	取决于 α	接近 O(1)	散列表

- **静态查找表**（只查不增删）→ 折半 / 分块；**动态查找表**（边查边插删）→ BST / AVL / B 树 / 散列。
- 折半查找的**判定树高 = $\lceil \log_2(n+1) \rceil$**；精确 ASL 须画判定树逐层数比较次数，属计算（交给题）。
- 折半两个硬前提：**表有序 + 顺序存储**。两者缺一不可。

查找这章 18 年全勤，五个 TOP KP（B 树 / AVL / 各查找比较 / 折半 / BST）轮流坐庄，**「哪种方法用在什么结构上、ASL 量级各是多少」是选择题送分点**。这张卡只锚汇总结论，具体 ASL 演算留给题和判定树。

最该钉死的三组对照：

- **要不要有序、要不要顺序存储**：顺序查找两者都不要求；折半两者都要求（有序 + 顺序存储，**链表做不了折半**，因为取不到 mid）；分块要求块间有序、块内可乱。
- **成功 ASL 的量级**：顺序是 (n+1)/2（线性），折半是 log 级，散列接近常数。折半成功 ASL **不要去背半精确的「$\approx \log_2(n+1)-1$」**——那是近似式会误导；要背的是「**判定树高 $\lceil \log_2(n+1) \rceil$**，精确值画判定树数出来」。
- **静态 vs 动态选型**：表一旦建好不再增删（静态）就用折半/分块；要频繁插入删除（动态）就走 BST/AVL/B 树/散列，避免每次重排有序表。

  折半查找的判定树构造、每个失败区间的比较次数、分块的最优块长推导、散列的成功/失败 ASL 计算，全是手算，交给题和上方可视化。卡里只锚「方法 × 适用结构 × ASL 量级」的对照。

B+ 树 低频 对比表

B 树 vs B+ 树对照（大纲只要基本概念）：

对比项	B 树	B+ 树
数据存储位置	所有结点都存数据	**仅叶结点存数据**
非叶结点作用	既是索引又存数据	**纯索引**，不存实际记录
关键字数 ↔ 子树数	n 个关键字 → **n+1** 棵子树	**n 个关键字 → n 棵子树**（相等）
查找路径	可能在任意层命中	**必须查到叶结点**
叶结点链表	无	**有**，叶间用指针串成有序链表（支持顺序遍历/范围查询）
关键字是否重复	不重复	内部结点关键字会在叶结点**再次出现**

- n 阶 B+ 树非根内部结点关键字数范围 **⌈n/2⌉ ~ n**。
- 数据库索引（MySQL InnoDB 等）多用 B+ 树：树更矮（I/O 少）+ 范围查询沿叶链表扫即可 + 查找路径稳定。

B+ 树是 B 树的范围查询优化版：把**所有数据都沉到叶结点、叶结点用链表串起来**，范围查询（找 100~200 之间所有记录）只需在叶链表上顺序扫，不用在树里反复上下跳。这张卡背 B/B+ 对照即可，大纲只要基本概念。

最该钉死的三个区别：

- **关键字数与子树数的关系**：B 树是「关键字 = 子树 − 1」，B+ 树是「关键字 = 子树」（相等）。这是命题组最爱挖的概念坑。
- **非叶结点存不存数据**：B 树每个结点都可存数据；B+ 树非叶结点只放关键字副本当索引，**真正的数据记录全在叶结点**。所以 B+ 树查找无论中途是否"命中"都**必须走到叶结点**才能取到数据。
- **叶结点链表**：B+ 树叶间有指针相连成有序链表，这是它支持高效范围查询/顺序遍历的关键，也是与 B 树最本质的差别；B 树没有这条链。

  B+ 树的插入分裂、删除合并、给定序列建树的具体形态都是手练，交给题和上方可视化。卡里只锚 B 树 vs B+ 树的结构对照表。

散列表-拉链法 低频 对比表

两类冲突处理法对照（拉链法 vs 开放定址法）：

对比项	拉链法（链地址法）	开放定址法
冲突处理	同义词挂到**同一槽位的链表**上	冲突就**探测下一个空位**（线性/二次/双散列）
装填因子 α	**可以 > 1**（链表长度不限）	**必须 < 1**（数据全存表内，表会满）
删除操作	直接删链表结点	只能**逻辑删除（打墓碑标记）**，不能真删
堆积（聚集）	**不产生**	**会产生**（线性探测最严重，非同义词也聚集）
空间	需额外指针	无指针开销

- **装填因子 α = n / m**（n=元素数，m=表长）；**α 越大冲突越多、ASL 越大**。
- 提高查找效率的正确手段：设计冲突少的散列函数 + 处理冲突时避免堆积（**单纯增大 α 反而更差**）。

散列查找想要接近 O(1)，代价是**冲突**——两个不同关键字算到同一地址怎么办。两大流派：拉链法「冲突的挂一条链」，开放定址法「冲突的往后探一个空位」。这张卡只背两类法的本质区别 + 装填因子定义，具体 ASL 计算和探测序列交给题。

最该钉死的三组辨析：

- **装填因子 α = n/m 的范围**：拉链法 α 可以 > 1（链表能无限挂），开放定址法 α 必须 < 1（位置存满就插不进）。无论哪种，**α 越大 → 冲突越多 → ASL 越大**。
- **删除方式不同**：拉链法直接摘链表结点；开放定址法**不能物理删除**（删了会截断后续同义词的探测路径），只能打"墓碑"标记做逻辑删除。
- **堆积只发生在开放定址**：拉链法不会堆积；开放定址的线性探测最严重——**非同义词也会争抢同一段连续空间（一次聚集）**，二次探测能缓解（仅同义词聚集）。所以「增大装填因子」不是提效手段，反而加剧冲突。

  拉链法/开放定址法的成功、失败 ASL 具体数值，线性/二次探测的探测序列，给定关键字序列画散列表，全是手算，交给题和上方可视化。卡里只锚两类法的本质区别 + α 定义与影响。

红黑树 冷门 硬结论数字

红黑树五条性质（裸结论，逐条背）：

- ① 每个结点**非红即黑**。
- ② **根结点是黑色**。
- ③ **叶结点（NIL 空结点）是黑色**（这里的叶指外部空结点，不是普通叶子）。
- ④ **红结点的孩子必为黑**（不存在两个连续的红结点，即"不红红"）。
- ⑤ 从**任一结点到其所有叶（NIL）结点的路径上，黑色结点数相同**（黑高相等）。

结论（由性质推得）：

- **最长路径 ≤ 2 × 最短路径**（最长=红黑交替、最短=全黑），故树高 h ≤ **$2\log_2(n+1)$**。
- 查找/插入/删除均 **O(log n)**。
- **插删比 AVL 旋转少**（插入最多 2 次旋转、删除最多 3 次），更适合**频繁修改**的场景。

红黑树是「平衡度」与「维护成本」的折中：不像 AVL 那样要求左右子树高度严格接近，**只要求任意一条路径上黑结点数相同**。代价是树可能比 AVL 高近一倍，但插入删除的旋转次数有上界，维护更便宜——所以 Linux 调度、Java TreeMap、C++ std::map 都选它。这张卡只到五条性质 + 结论，调整过程不展开。

最该钉死的两个点：

- **性质③的"叶"是 NIL 空结点**：不是平常说的「没有孩子的内部结点」，而是查找失败到达的外部空结点。算黑高、判断性质时一定把 NIL 当黑结点画出来，否则容易数错。
- **由④⑤推出「最长 ≤ 2× 最短」**：最短路径全是黑结点，最长路径红黑交替（红结点不能连续，所以红最多占一半），两条路径黑结点数又必须相等，于是最长不超过最短的 2 倍——这就保证了 O(log n) 的高度。

  插入后的着色调整、删除后的旋转修复（叔结点颜色分情况、变色 vs 旋转）都是手练且超出大纲要求，**这张卡只背到五条性质，不展开插入删除调整过程**。要看调整动画点上方可视化。

排序 2 张

各排序算法比较 中频 硬结论数字

排序总表（时间 / 空间 / 稳定性，考前默写）：

算法	最好	平均	最坏	空间	稳定性
直接插入	O(n)	O(n²)	O(n²)	O(1)	稳定
折半插入	O(n log n)	O(n²)	O(n²)	O(1)	稳定
冒泡	O(n)	O(n²)	O(n²)	O(1)	稳定
简单选择	O(n²)	O(n²)	O(n²)	O(1)	**不稳定**
希尔	—	约 O(n^1.3)	O(n²)	O(1)	**不稳定**
快速	O(n log n)	O(n log n)	**O(n²)**	O(log n)	**不稳定**
堆	O(n log n)	O(n log n)	O(n log n)	O(1)	**不稳定**
归并	O(n log n)	O(n log n)	O(n log n)	**O(n)**	稳定
基数	O(d(n+r))	O(d(n+r))	O(d(n+r))	O(r)	稳定

四句口诀（钉死）：

- 稳定性 → **快选希堆不稳定**，其余（插入/折半插入/冒泡/归并/基数）全稳定。
- 最好 O(n) → **插入、冒泡**（基本有序时一趟近似扫完）。
- 最坏也 O(n log n) → **归并、堆**；快排最坏退化 O(n²)。
- 空间非 O(1) → **快排 O(log n)、归并 O(n)、基数 O(r)**，其余 O(1)。
- **唯一稳定的 O(n log n) 比较排序 = 归并。**

三组辨析结论（选择题高频）：

- 趟序特征：**冒泡 / 简单选择 / 快速 / 堆排序，每趟结束至少确定 1 个元素的最终位置**（"第 k 趟后哪些已就位"靠它判）——快排定的是**枢轴**（不一定是极值），另外三个定的是当前极值；**插入 / 希尔 / 归并不行**，插入排序的"一趟"只是把第 k+1 个元素插进前 k 个，未必定位最终位置。
- 初态敏感性：**插入 / 冒泡 / 快排 对初始序列敏感**（已基本有序时插入冒泡快、快排反而退化）；**简单选择 / 归并 / 堆 / 基数 不敏感**（比较次数与初态无关）。
- 稳定性反例锚点：简单选择对 5 5 3 —— 第一趟选最小的 3 与第一个 5 交换，第一个 5 跑到第二个 5 后面，故不稳定；希尔因**跨组跳跃交换**打乱相同关键字相对位置，故不稳定。

排序通论（概念前置）： 内排序=数据全在内存 / 外排序=数据在外存；稳定性定义=相同关键字排序前后相对位置不变；比较类下界 O(n log n)（判定树 + Stirling），非比较类（基数 / 桶）按位分配可突破下界。

排序对比表是排序章每年至少送 1 道选择题的"钉子户"，整张表必须脱手默写。下面只讲怎么把结论记牢、记不混，**具体每趟排成什么形态、某次比较交换的中间值一律交给题和可视化**——背中间值是浪费。

稳定性怎么记——口诀「快选希堆不稳定」覆盖全部 4 个不稳定算法（快速、简单选择、希尔、堆），剩下 5 个全稳定，正着背反着背都行。判定本质：排序过程中是否可能把两个相同关键字元素的相对顺序颠倒。不稳定只要举一个反例，稳定要严格证明，所以记住反例比记结论更牢——简单选择记 5 5 3 ，希尔记"跨组跳跃"。

时间复杂度怎么记——先记三个"最坏也 O(n log n)"的优等生：归并、堆。再记两个"最好能到 O(n)"的：插入、冒泡（它们对基本有序数据敏感，几乎不动）。最危险的陷阱是快排：平均最快 O(n log n)，但数据已有序时退化到 **$O(n^2)$**，且空间不是 O(1) 而是递归栈的 O(log n)。

空间复杂度怎么记——默认 O(1)，只挑出三个例外：快排 O(log n)（递归栈）、归并 O(n)（要辅助数组）、基数 O(r)（r 个桶）。

三组辨析为什么常考——①趟序：题目爱问"每趟能确定一个元素最终位置的是哪些"，答案锁定交换/选择类里的冒泡、快排、简单选择、堆（2012 真题原题）；插入、希尔和归并不行。快排容易漏——它每趟把**枢轴**放到最终位置，只是这个位置上的值不一定是极值。②初态敏感性：问"对基本有序序列效率最高的"答直接插入 O(n)；问"比较次数与初始序列无关的"答简单选择（恒 $n(n-1)/2$）和归并、堆。③稳定性反例：能当场写出反例就不会被"它一定稳定吧"带偏。

> 各排序的具体排序过程（每趟形态、partition 划分点、建堆调整、二路归并的中间序列）属于手算，**只背时间/空间/稳定性/趟序/敏感性这几格结论**，过程点「展开博客精讲」或下方"排序算法对比"可视化逐帧看。

外部排序 低频 定义锚点

外部排序 = 生成初始归并段 + 多路归并（数据量超过内存时的唯一方案）。

- 两阶段：① 把大文件分块读入内存、内部排序后写回，生成 m 个有序**初始归并段**；② 对 m 个段做 **k 路归并**，多趟合并到整体有序。
- **性能瓶颈是磁盘 I/O 次数，不是比较次数**；I/O 次数 ∝ 归并趟数。
- 归并趟数 **S = ⌈log_k(m)⌉**（m=初始归并段数，k=归并路数）。**k 越大、m 越小，趟数越少。**

减少趟数的两条途径：

- **增大归并路数 k** → S = ⌈log_k(m)⌉ 随 k 增大而减小。
- **减少初始归并段数 m** → 用**置换-选择排序**生成平均长度 2M 的更长归并段（M=内存工作区大小），m 随之减小。

两个配套优化（背作用，不背手算）：

- **败者树**：k 路归并每次选最小值，直接比较要 k−1 次，败者树降到 **⌈$\log_2 k$⌉** 次（k 越大优势越明显）。
- **置换-选择排序**：生成更长（平均 2M）的初始归并段，从源头减少 m。

外部排序是排序章的收官内容，近 3 年加权上升（2023 还出了 10 分大题）。它的世界观和内部排序完全不同：**衡量代价的不是比较次数，而是磁盘读写（I/O）次数**——因为外存比内存慢几个数量级。所有优化都围绕"少读写磁盘"。这张卡只锚定义和定性结论，**具体趟数、虚段个数、败者树调整路径这些数值全是计算/手算，交给题**。

整个流程一句话——大文件放不进内存，就先切块、每块在内存里排好序写回磁盘（这些有序块叫"初始归并段"），再用多路归并一趟趟把它们合并成一个完整有序文件。

为什么盯着"归并趟数"——每归并一趟，所有数据都要从磁盘读一遍、写一遍，I/O 次数和趟数成正比。趟数公式 **S = ⌈log_k(m)⌉** 是核心：要么把底数 k 调大（多路归并），要么把段数 m 调小（每段更长），都能压低趟数。这就是外部排序两大优化方向的由来。

败者树解决什么——k 路归并每输出一个元素，都要从 k 个段的当前元素里挑最小值。直接两两比要 k−1 次，k 一大就拖慢内部处理。败者树把每次选最小的比较次数降到 ⌈$\log_2 k$⌉（比如 8 路：直接比 7 次，败者树只 3 次），让"增大 k"不会被内部比较开销吃掉。

置换-选择排序解决什么——普通切块每段长度等于内存大小 M；置换-选择能生成平均 2M 的更长段，段数 m 直接砍半，配合公式 S=⌈log_k(m)⌉ 进一步降趟数。它生成的段长通常不等，于是又引出"最佳归并树"（k 叉哈夫曼树 + 虚段补充）——那部分是大题里的计算，**只记"作用 = 生成更长归并段"，怎么补虚段、构造哪棵树交给题**。

> 归并趟数、虚段个数、败者树某次调整后的结构、最佳归并树的 WPL，这些都是计算/手算，记忆卡不背。结论只锚到"外排=归并段+多路归并、瓶颈在 I/O、k 越大趟数越少、败者树降比较、置换-选择生成更长段"。过程点「展开博客精讲」或下方可视化逐帧看。

扫码回站内背这套卡
站内按记忆曲线排期，到期自动进「今日待背」